

Histogramas

Discente: Pedro Cézar S. Ferreira

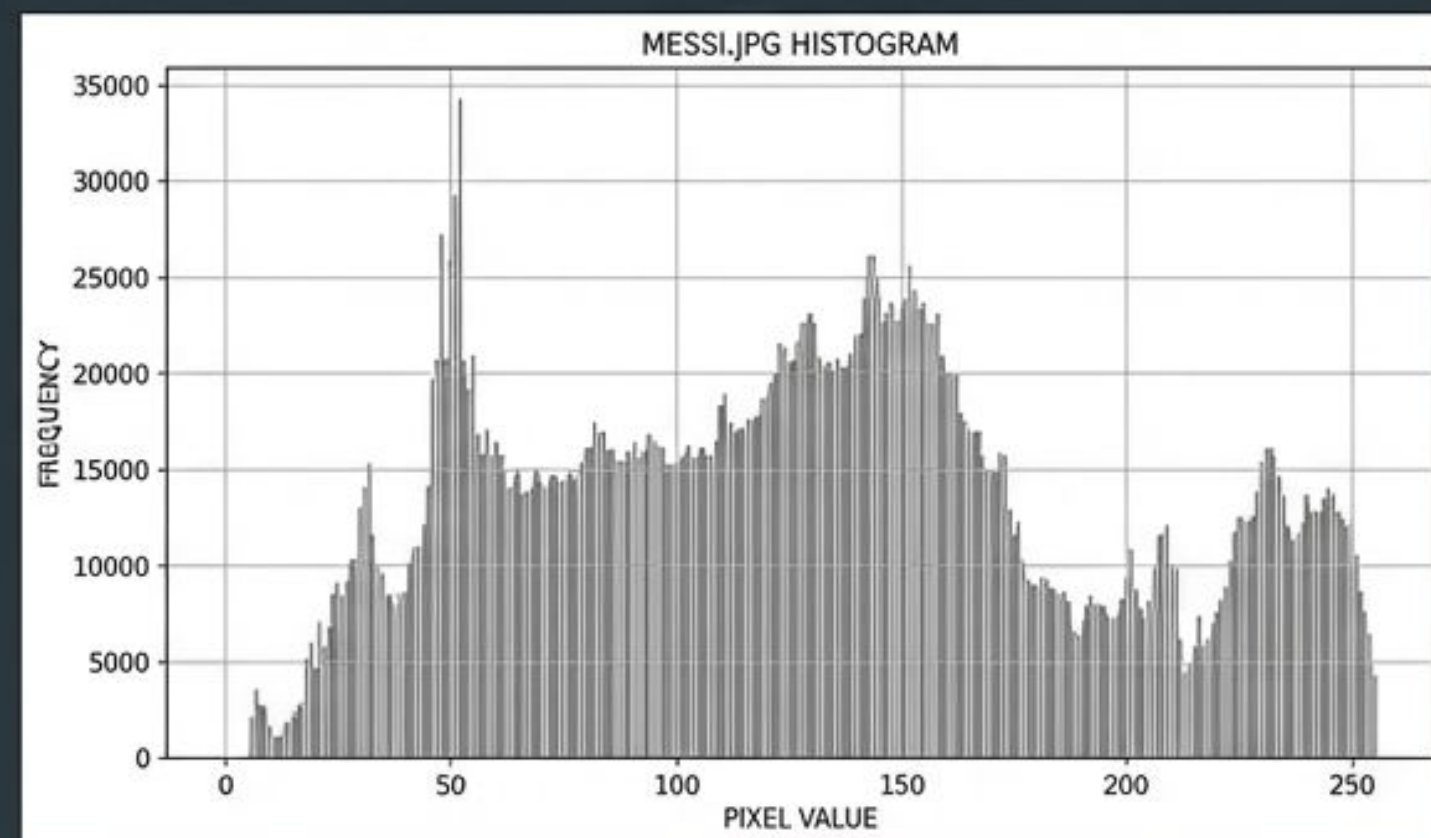
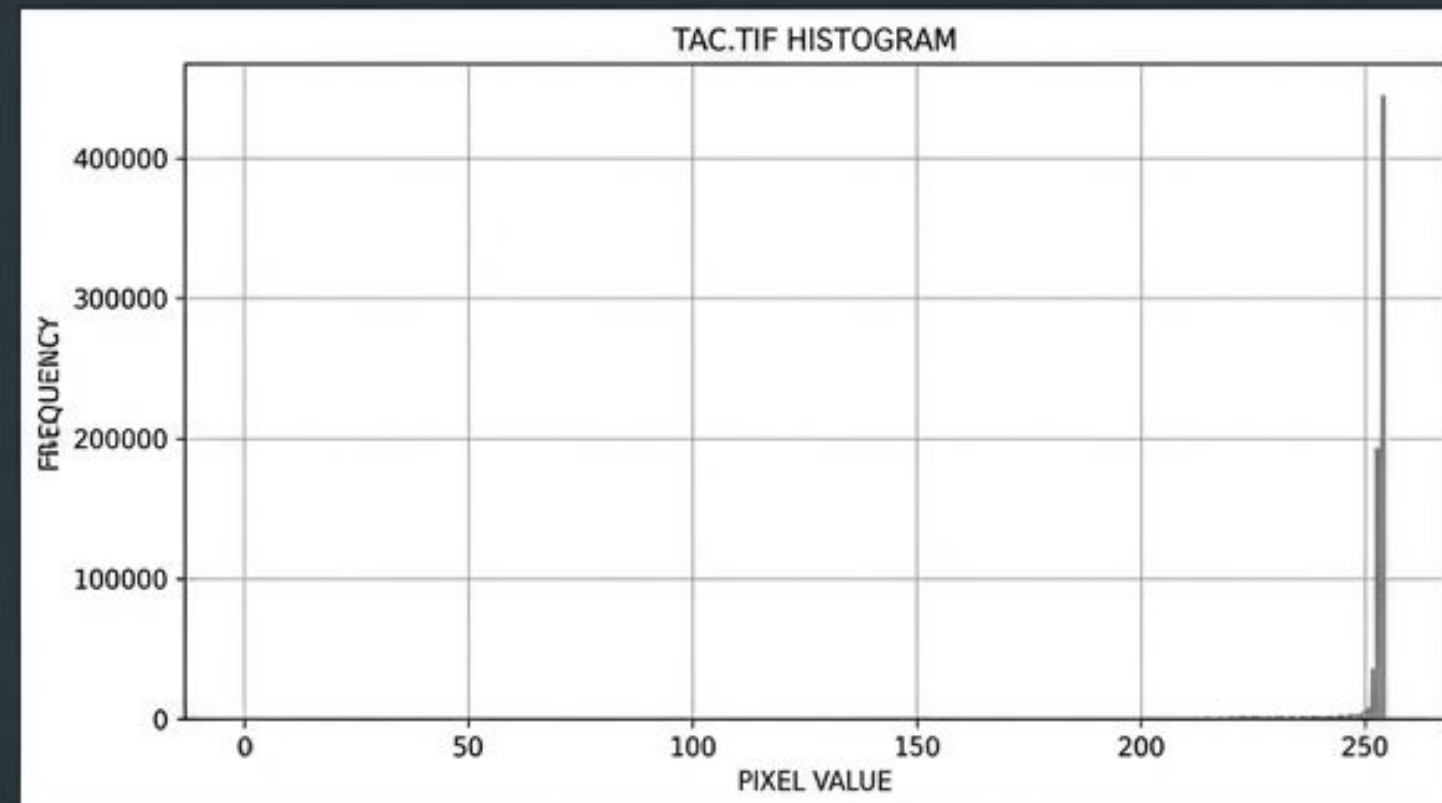
Professor: Prof. Dr. Cassio Dener Noronha Vinhal

Lab. 2

Módulo I: O Conceito de Binning

O binning é o agrupamento das intensidades dos pixels em faixas (bins). A quantidade define o detalhe: menos bins simplificam o gráfico, mais bins revelam detalhes ou destacam ruídos.

```
def HISTOGRAM(IMG_DATA, BINS=256):  
    HIST = np.zeros(BINS, dtype=np.int64)  
    for PIXEL_VALUE in IMG_DATA.ravel():  
        HIST[PIXEL_VALUE] += 1  
    return HIST
```



A Resolução do Histograma

Com **poucos bins**, **perde-se o detalhe** visual; intensidades diferentes ficam intensamente misturadas.

Com **muitos bins**, o gráfico pode se **fragmentar**, destacando excessivamente os ruídos da imagem.

Reflexão sobre Arquitetura

Diferenças conceituais entre abordagens:

- **Python (Implementada)**: Mais direta, interativa e didática (uso de numpy e ravel).
- **Java (Burger)**: Mais estruturada e estritamente orientada a objetos. O conceito matemático base permanece o mesmo.

Módulo II: A Matriz de Variantes do Histograma

Histograma Absoluto	Histograma Normalizado	Histograma Cumulativo
O que mostra: A quantidade real de pixels em cada nível de intensidade. Uso Primário: Contagem base e compreensão do volume de dados brutos.	O que mostra: Proporção ou probabilidade de cada intensidade (Pixels do nível / Total de pixels). Relacionamento: Atua como a PDF (Função de Distribuição de Probabilidade) discreta da imagem.	O que mostra: Soma acumulada das frequências até cada nível. Uso Primário: Equalização, ajuste de contraste e contagem rápida de pixels abaixo de um limiar.

Calculando Probabilidade e Acúmulo

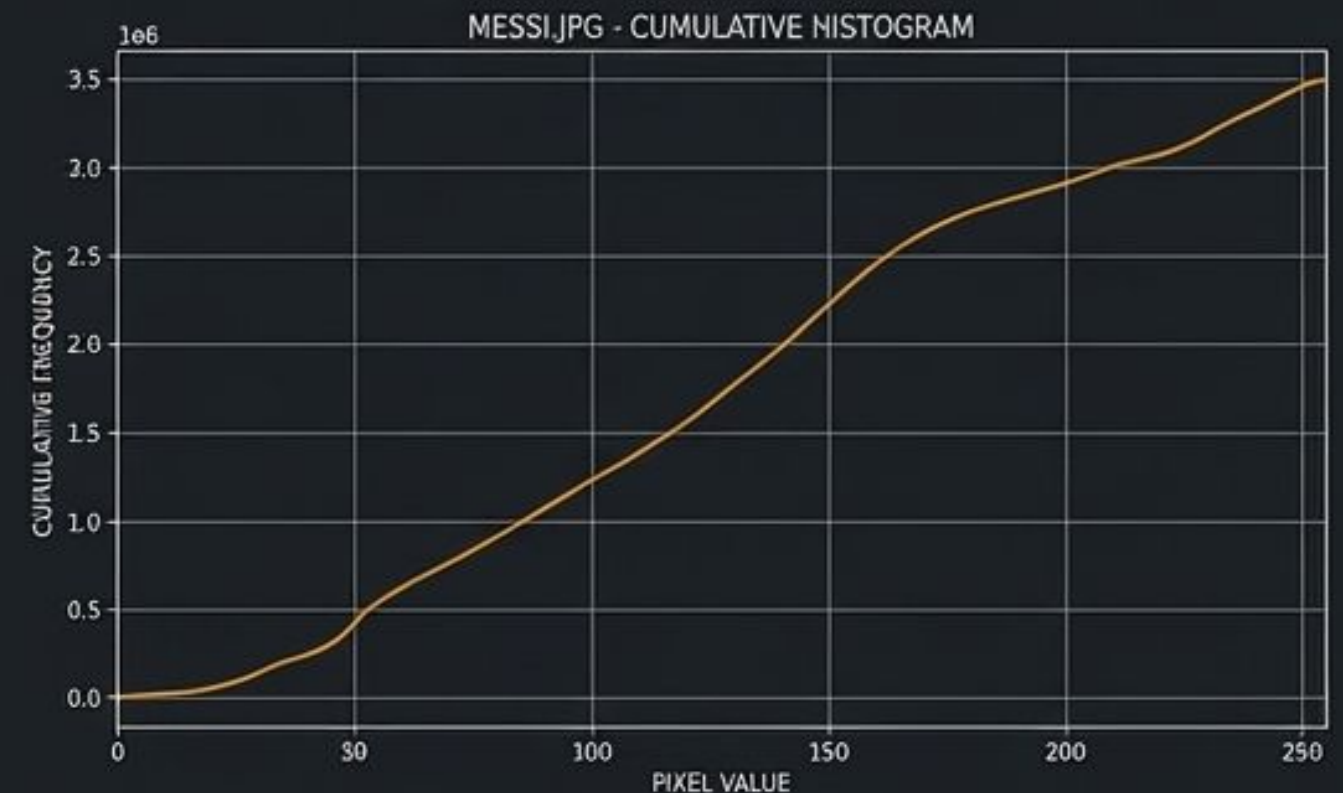
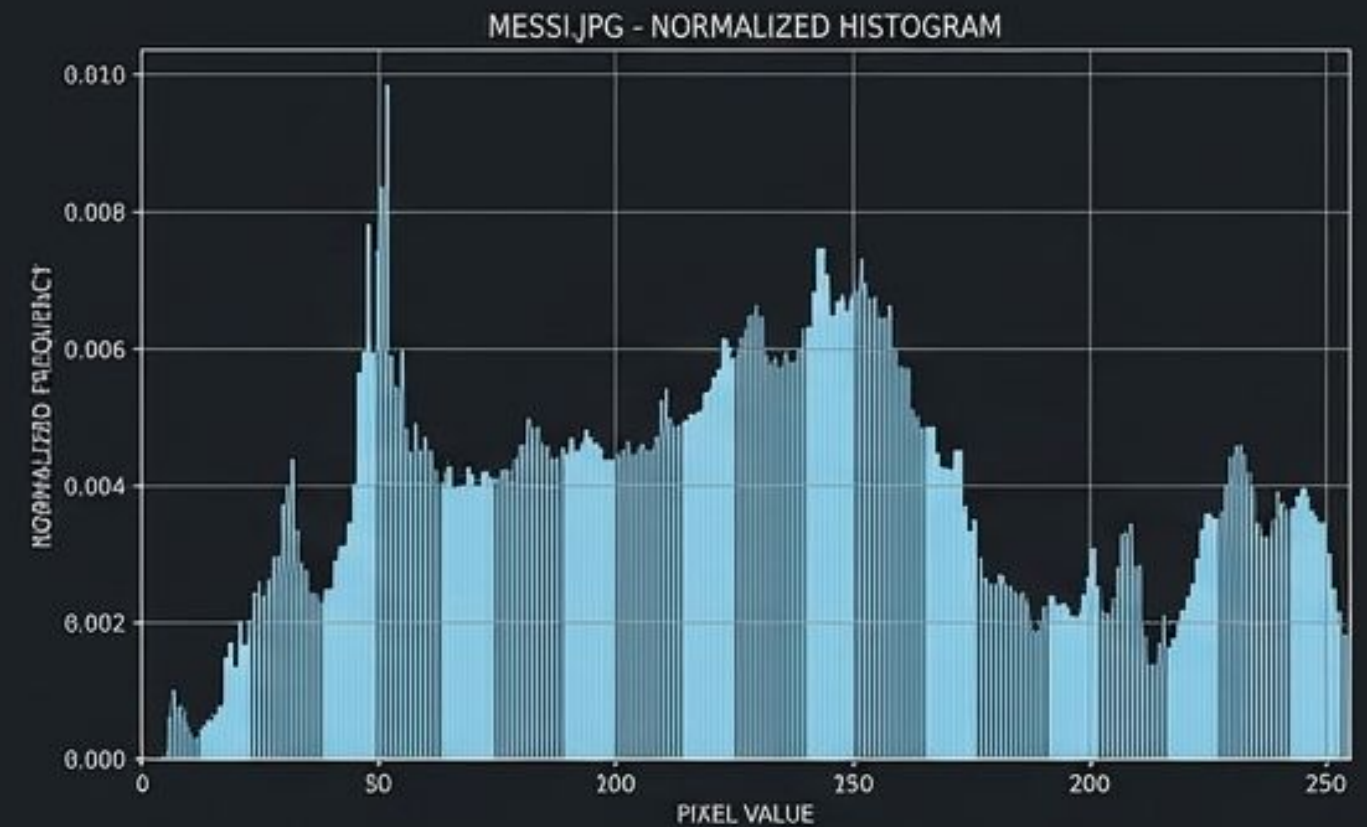
```
TOTAL_PIXELS = sum(HIST_DATA)
```

```
# Normalizado: Probabilidade (PDF)
```

```
NORMALIZED_PROB = HIST_DATA[i] / TOTAL_PIXELS
```

```
# Cumulativo: Soma em cascata
```

```
CUMULATIVE_SUM += HIST_DATA[i]
```



Observe a curva de crescimento contínuo no cumulativo em oposição aos picos de probabilidade no normalizado.

Regras de Diagnóstico

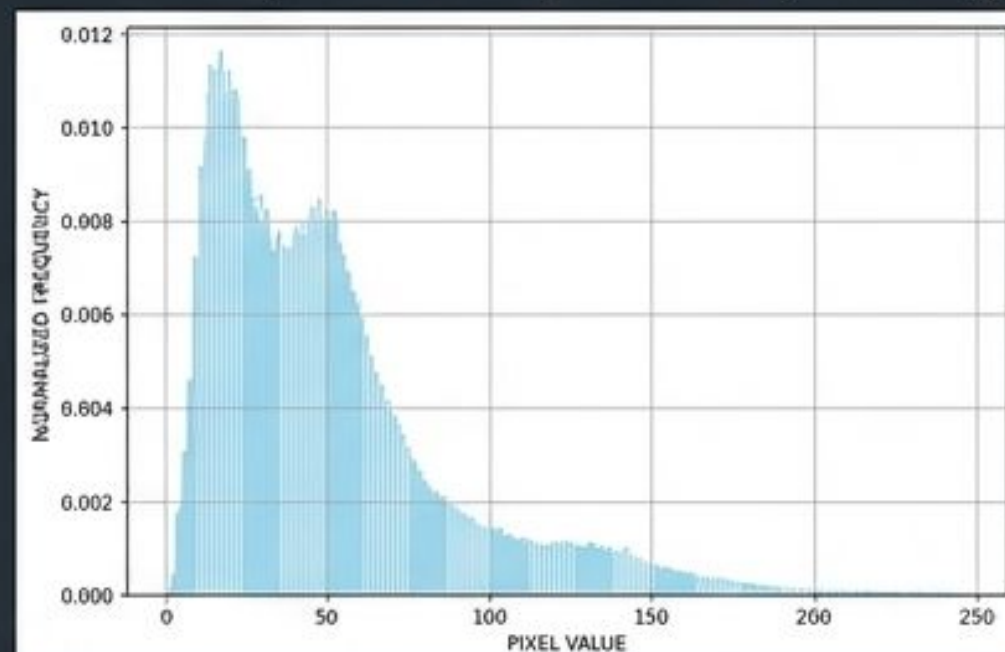
Módulo III: Guia de Diagnóstico (Luz e Sombra)

O histograma reflete a distribuição de intensidade de luz da imagem.

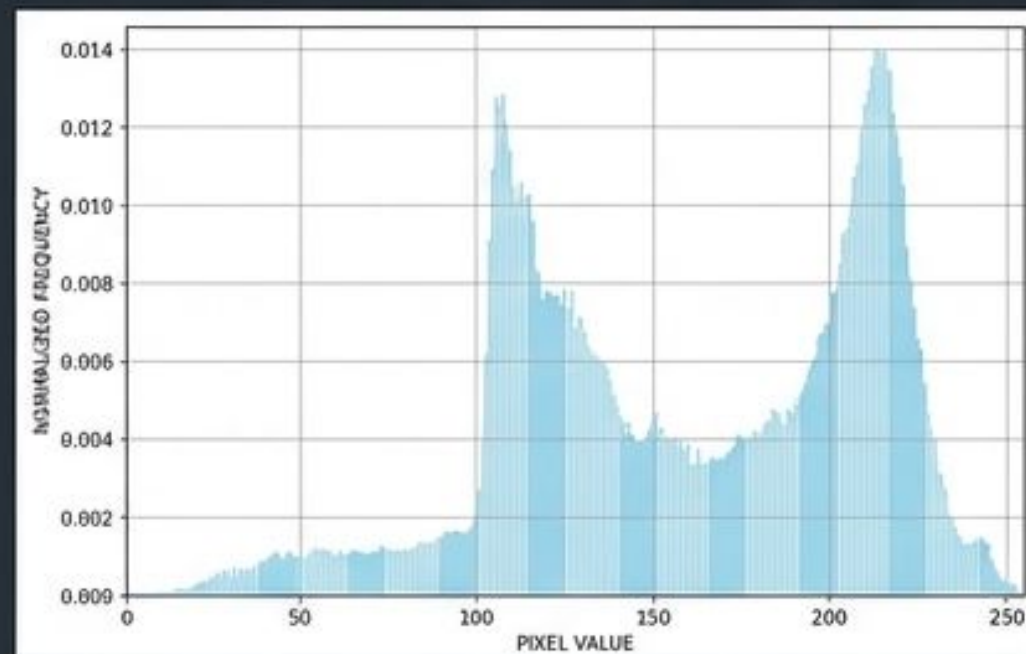
- **Imagens Escuras:** A massa de dados concentra-se à esquerda (tons baixos).
- **Imagens Claras:** O histograma desloca-se e concentra-se à direita (tons altos).

Evidência

Concentração à Esquerda (Dark.jpg)



Distribuição Balanceada



Regras de Diagnóstico

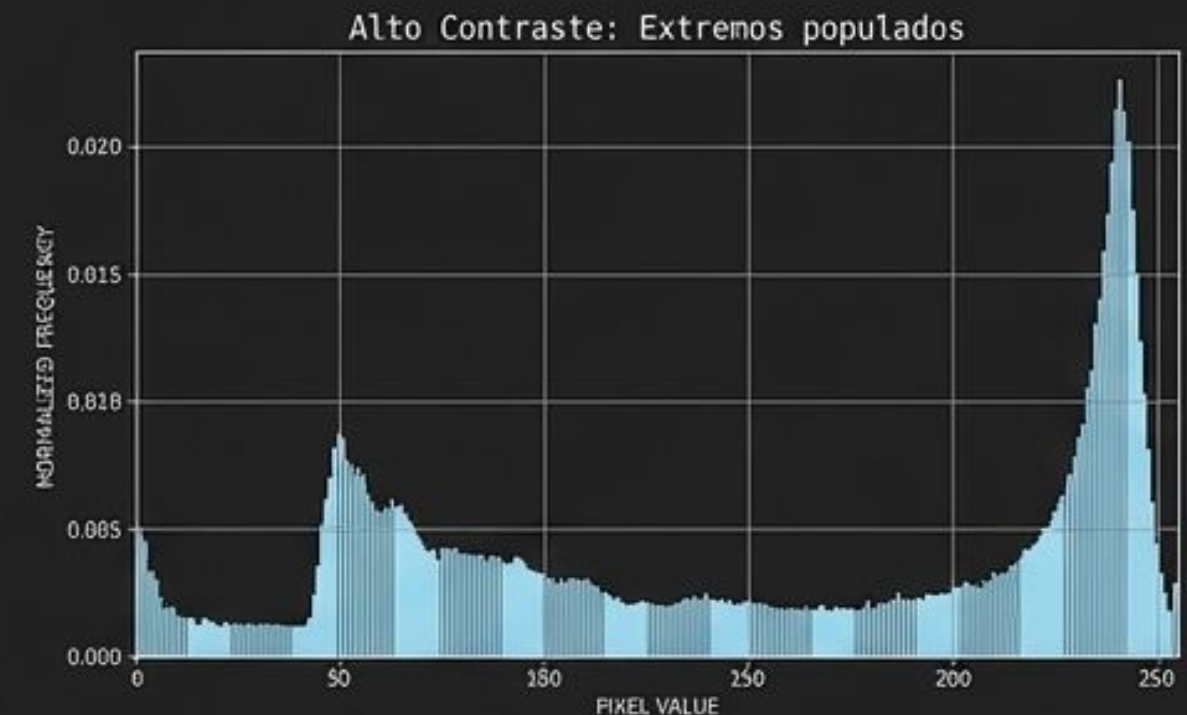
Guia de Diagnóstico (Contraste e Manipulação)

Baixa variação de intensidade achata o gráfico em uma faixa estreita, indicando baixo contraste.

Detecção de Manipulação

O histograma sozinho não prova manipulação, mas fornece indícios. Picos anômalos, lacunas abruptas ou distribuições incomuns sugerem alterações artificiais (ex: aumento de contraste forçado ou recortes).

Evidência



“O dynamic range é a faixa de intensidades que a imagem utiliza, do tom mais escuro ao mais claro.”

Baixo Dynamic Range

Histograma concentrado em uma faixa pequena. Geralmente indica baixo contraste geral.

Alto Dynamic Range

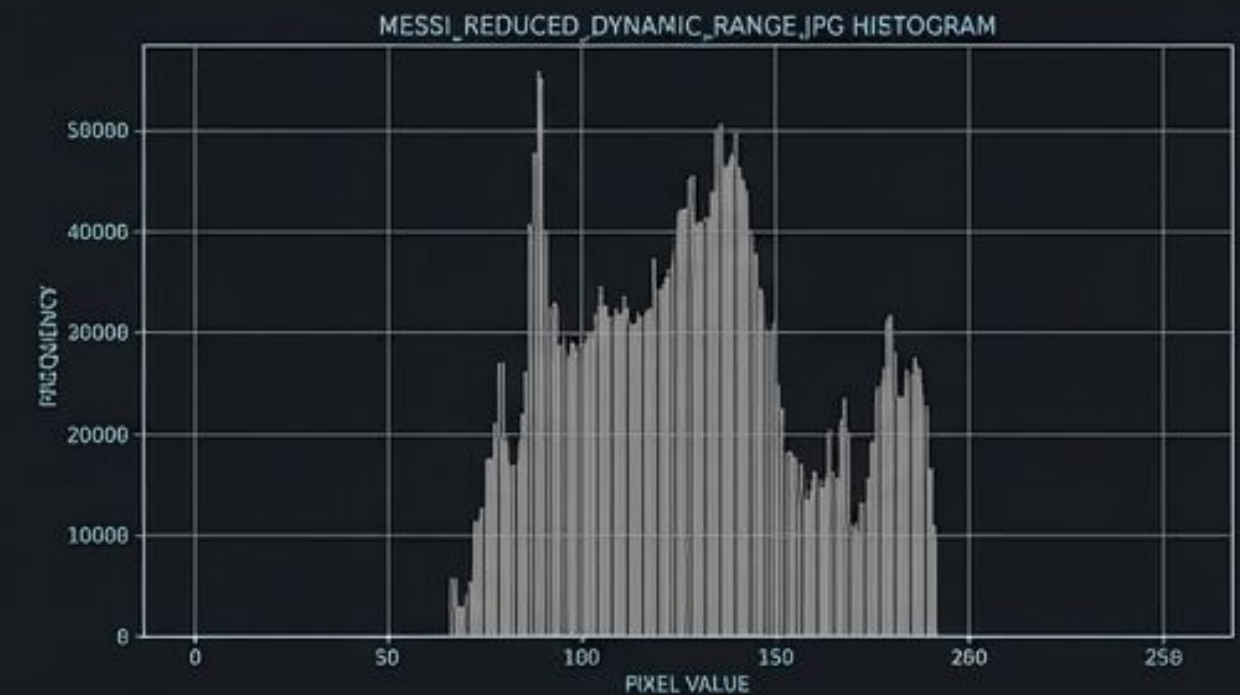
Histograma espalhado e distribuído, utilizando uma faixa maior de níveis de cinza. Maior contraste inerente.

Compressão Matemática

Reduzindo o Range Artificialmente

```
1 # Centraliza em 128 e comprime a distância pela metade
2 REDUCED_RANGE = 128 + 0.5 * (IMG_GRAY - 128)
3 REDUCED_RANGE = np.clip(REduced_RANGE, 0, 255)
4
5
```

Impacto Visual



A largura da distribuição visualiza matematicamente o dynamic range disponível.

Decomposição RGB

Módulo V: O Salto para Matrizes de Cor

Em imagens coloridas, o histograma é calculado isolando e processando independentemente cada canal de cor.

```
1 HIST_B = np.zeros(BINS, dtype=np.int64)
2 HIST_G = np.zeros(BINS, dtype=np.int64)
3 HIST_R = np.zeros(BINS, dtype=np.int64)
```

Visual Output



Sobreposição visual das frequências dos canais Vermelho, Verde e Azul.

Módulo VI: Implementação Manual vs. Bibliotecas (OpenCV)

Implementação Manual

Vantagens:

Facilita o entendimento do algoritmo base e garante controle total sobre cada etapa do processo.

Desvantagens:

Mais trabalhoso e sujeito a erros no tratamento de dados.

Performance:

Lento para imagens grandes devido a loops iterativos puros em Python.

`cv2.calcHist()`

Vantagens:

Rápido, confiável e extremamente prático para pipelines de produção.

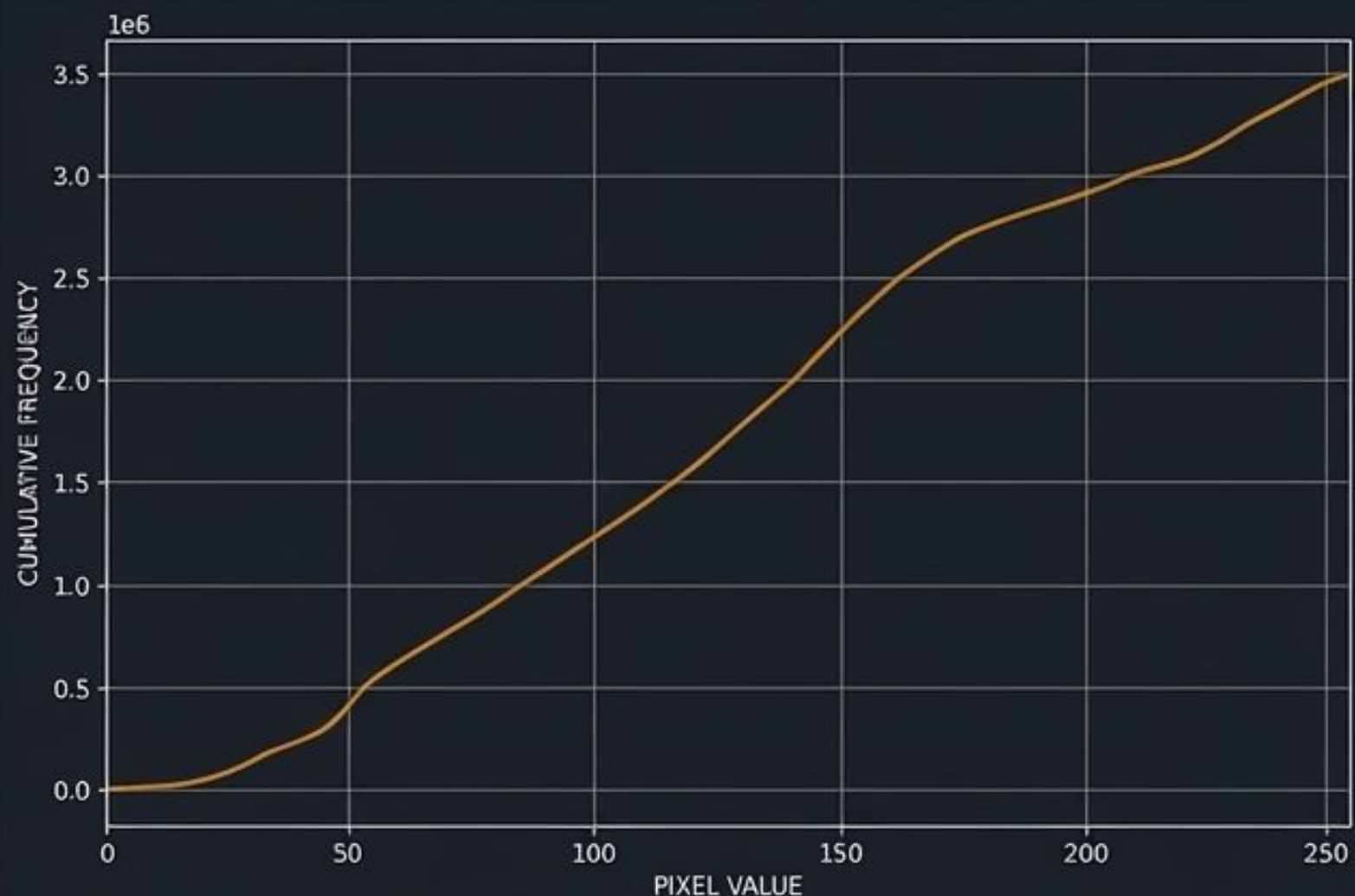
Desvantagens:

Age como uma 'caixa preta', escondendo parte da lógica matemática interna do usuário.

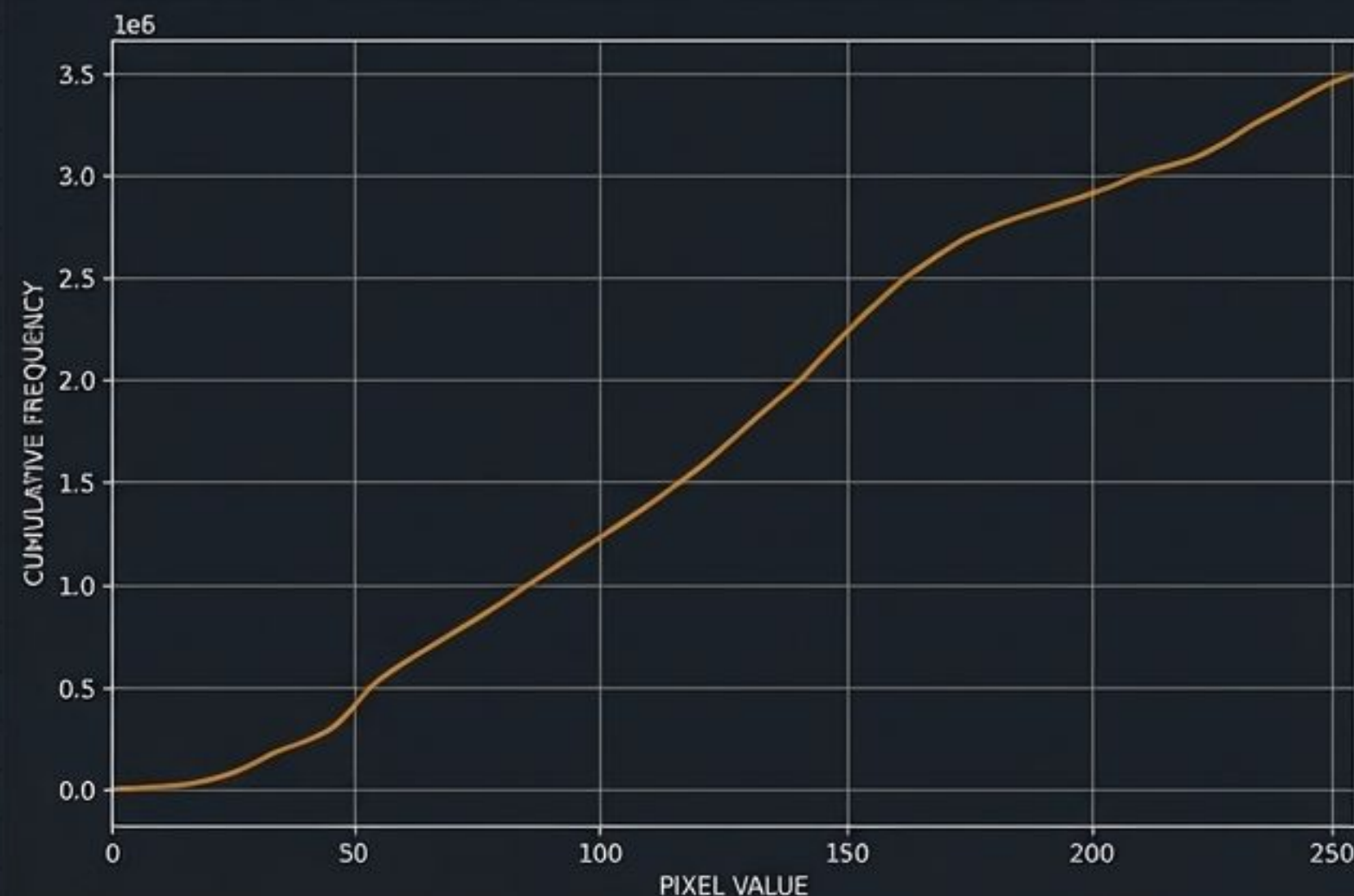
Performance:

Altamente eficiente. Otimizado internamente (C/C++) para manipulação de grandes arrays e matrizes de imagem.

Output Manual (HISTOGRAM)



Output OpenCV (cv2.calcHist)



Conclusão técnica: Se a lógica interna for respeitada, o resultado matemático é idêntico. A escolha da ferramenta dita apenas a velocidade e a manutenibilidade do código.